

Go Benchmark Baseline Registry

Revision: 1 Last modified: 2026-06-23T00:00:00Z

Audit gap **G5** (docs/research/test-coverage-audit-20260623/): 7 Go benchmarks exist but had no baseline / regression registry. This directory is that registry; tests/regression/guard_benchmark_baseline.sh is the regression guard.

Benchmarks covered (7, two packages)

Package	Benchmark	What it measures
internal/ api	BenchmarkHealthz	GET /healthz through the real <code>Server.Router()</code> (no auth)
internal/ api	BenchmarkGroupCreate	POST /api/v1/groups authenticated create path (auth + repo write)
internal/ api	BenchmarkGroupList	GET /api/v1/groups list path with 20 pre-seeded groups
internal/ api	BenchmarkClientUpdateNoDeployment	authenticated device GET /api/v1/client/update fast path → 204
internal/ store	BenchmarkMemoryCreateGroup	in-memory <code>Repository.CreateGroup</code> hot path
internal/ store	BenchmarkMemoryFindDelta	<code>Repository.FindDelta</code> lookup over 100 deltas (zero-alloc path)
internal/ store	BenchmarkMemoryListAudit	<code>Repository.ListAudit</code> over 200 audit entries, limit 50

Baseline files

- `baseline/internal_api.txt` — raw `go test -bench` output, `-count=6`
- `baseline/internal_store.txt` — raw `go test -bench` output, `-count=6`

These are the **raw** `go test -bench=.` `-benchmem` `-count=N` outputs (NOT pre-summarized). `benchstat` consumes raw output to compute medians + a distribution-aware A/B comparison, so the registry stores raw runs.

Captured on: Apple M3 Pro, darwin/arm64, go1.26.2.

Regenerate the baseline

When an intentional performance change lands (and is reviewed/approved), refresh the baseline:

```
cd server
go test -run='^$' -bench=. -benchmem -count=6 ./internal/api/
> ../docs/benchmarks/baseline/internal_api.txt
go test -run='^$' -bench=. -benchmem -count=6 ./internal/store/
> ../docs/benchmarks/baseline/internal_store.txt
```

Commit the refreshed baseline in the SAME commit as the perf change, with a note saying why the numbers moved (§11.4.6 — no silent baseline drift).

Compare a fresh run vs the baseline (manual)

```
benchstat docs/benchmarks/baseline/internal_api.txt /tmp/
bench_new.txt
```

benchstat prints the per-benchmark % delta and a p value / ~ marker. ~ means “no statistically significant difference given the noise.”

Install benchstat

```
go install golang.org/x/perf/cmd/benchstat@latest # → $(go env
GOPATH)/bin
```

Threshold calibration (§11.4.6 — measured, NOT hardcoded-from-literature)

Go microbenchmarks are inherently noisy on a shared developer laptop. Across the 6-iteration baseline runs (identical code, same machine, same build) benchstat reported these *intrinsic* variances:

Metric	Observed variance (identical code)	Implication
ns/op (sec/op)	16 % – 58 % (e.g. MemoryListAudit ±58 %, ClientUpdate ±44 %)	wall-clock is noisy → wide threshold needed
B/op	mostly ±0–1 %, one outlier ±23 % (MemoryCreateGroup map realloc jitter)	mostly stable
allocs/op		

Metric	Observed variance (identical code)	Implication
	± 0 % on every benchmark	rock-stable $\rightarrow$ tight threshold, primary signal

The guard therefore uses **two signals**:

1. **allocs/op — primary, tight.** allocs/op is deterministic (± 0 % observed). The guard FAILs if any benchmark's median allocs/op grows by **> 25 %** vs baseline. A real alloc regression (a new allocation in a hot path) is exactly what this catches, and the 25 % band tolerates the one B/op outlier without crying wolf.
2. **ns/op — secondary, wide.** Wall-clock is dominated by machine noise. The guard FAILs only on a **> 100 %** ($2\times$) median ns/op regression — chosen to sit well above the worst observed intrinsic variance (58 %) so normal jitter never trips it, while a genuine $2\times$ -slower regression still fails. This is a coarse smoke signal, not a precise gate; the precise gate is allocs/op.

Honest boundary (§11.4.6): on a noisy shared laptop, ns/op cannot give a tight gate without flaking. The guard is deliberately conservative on ns/op (won't flake) and tight on allocs/op (the deterministic signal). For a precise CPU-time gate you need a quiet dedicated runner + benchstat p-values; that is documented here as the known limitation, not faked.